

To understand how L1 and L2 regularization physically force weights to zero, we must look at how their respective penalties alter the gradient descent update rule.

When we apply regularization, we add a penalty term to our standard data loss L_{data} . The optimizer must now minimize a new total loss function:

$$L_{total} = L_{data}(w) + \lambda \cdot \text{Penalty}(w)$$

where λ controls how aggressively we enforce the penalty.

Here is the mathematical breakdown of how this changes the optimizer's behavior for a single weight, w .

L2 Regularization: Multiplicative Shrinkage (Weight Decay)

L2 regularization penalizes the **squared magnitude** of the weights. The penalty term is mathematically defined as $\frac{1}{2}w^2$ (the $\frac{1}{2}$ is added purely to make the derivative cleaner).

To find out how this affects the update step, we take the derivative of the total loss with respect to w :

$$\frac{\partial L_{total}}{\partial w} = \frac{\partial L_{data}}{\partial w} + \lambda w$$

When we plug this new gradient into the standard Gradient Descent update rule ($w_{t+1} = w_t - \eta \nabla w_t$), we get:

$$w_{t+1} = w_t - \eta \left(\frac{\partial L_{data}}{\partial w_t} + \lambda w_t \right)$$

By rearranging the terms, the true mechanical effect of L2 is revealed:

$$w_{t+1} = (1 - \eta\lambda)w_t - \eta \frac{\partial L_{data}}{\partial w_t}$$

The Intuition:

Before the optimizer even applies the data's gradient, it multiplies the current weight by a

fraction strictly less than 1 (e.g., 0.99). L2 acts like a continuous tax percentage. If $w = 100$, the tax is large. As w shrinks to 0.01, the tax becomes microscopic. Because the penalty shrinks proportionally with the weight, it continuously pushes the weight close to zero, but practically never reaches exactly zero.

L1 Regularization: Constant Subtraction (Sparsity)

L1 regularization penalizes the **absolute value** of the weights. The penalty term is defined as $|w|$.

The derivative of the absolute value function $|w|$ is the **sign function**, which outputs $+1$ if the weight is positive, and -1 if the weight is negative.

$$\frac{\partial L_{total}}{\partial w} = \frac{\partial L_{data}}{\partial w} + \lambda \text{sign}(w)$$

Plugging this into the Gradient Descent update rule yields:

$$w_{t+1} = w_t - \eta \frac{\partial L_{data}}{\partial w_t} - \eta \lambda \text{sign}(w_t)$$

The Intuition:

Unlike L2, which takes a percentage, L1 subtracts a **constant absolute amount** ($\eta\lambda$) from the weight at every single step, regardless of how big or small the weight is. If the weight is positive, it pushes it down by a fixed step. If the weight is negative, it pushes it up by a fixed step.

Because this penalty does not scale down as the weight gets smaller, a tiny weight (e.g., $w = 0.001$) will get hit by the exact same massive penalty as $w = 100$. This constant downward pressure easily drives small, irrelevant weights completely through zero, effectively deleting the feature from the model.